

Day 56: Sequential Data

MD Mutasim Billah | 52-Week Data Science → ML/AI Roadmap | Week 9: Sequence Data & RNNs I
(Day 1 of 7)

0. Where Today Fits

Week 9 begins a new theme: sequence data, where ORDER carries meaning. Every network this series has built so far -- dense (Days 36-41), CNN (Days 42-55) -- either flattens its input into a fixed-size vector or slides a fixed-size window over it. Today builds a real, honest demonstration of exactly where and why that breaks down, using a synthetic but genuine "order matters" task.

Start here (no prior knowledge assumed):

Today's task: a length-30 sequence contains one HIGH spike (+3) and one LOW spike (-3) at two positions; the label is 1 if the high spike occurs BEFORE the low spike in time, 0 otherwise. The actual VALUES present in the sequence are identical either way -- only their relative ORDER decides the label. This is a clean stand-in for real sequential structure: the same two events, different meaning depending on which came first (like "price went up then down" vs. "down then up").

Watch: [Recurrent Neural Networks \(RNNs\), Clearly Explained!!!](#)

Watch: [CNN Receptive Field | Deep Learning Animated](#)

1. Failure Mode #1 -- PlainFCNet Has No Position Invariance

SYSTEM DIAGRAM

A dense network's first-layer weight matrix W_1 has shape (T, hidden) -- ONE ROW PER INPUT POSITION. Row t only ever receives a gradient signal when input position t actually carries information during training. Train with both spikes confined to positions 0-14, and rows 15-29 never learn a useful detector -- there is no mechanism by which a pattern learned at one position transfers to another.

ACTUAL OUTPUT

```
train_acc (seen positions): 1.0000
test_acc (seen positions, held out): 0.8875
test_acc (SHIFTED positions 15-29): 0.5500
```

PlainFCNet genuinely learns the rule for positions 0-14 (train_acc=1.000, held-out seen-position test_acc=0.888) -- but on the IDENTICAL rule applied to positions 15-29, accuracy collapses to 0.550, chance level for 2 classes. Nothing about the task changed; only where the pattern occurs did.

2. The Fixed-Length Limitation -- No Training Needed

A second, purely architectural point: a dense network's input size is fixed at construction time. It cannot even ACCEPT a sequence of a different length, regardless of how well trained it is.

ACTUAL OUTPUT

ValueError raised, as expected: PlainFCNet was built for sequence length 30, but received length 20 -- a dense network's input size is fixed at construction time.

3. Failure Mode #2 -- Conv1DNet's Capped Receptive Field

SYSTEM DIAGRAM

Conv1DNet (one 1D conv layer, reusing Day 53's conv2d_forward_mc/backward_mc via a reshape trick, then global average pooling) DOES have translation invariance via weight sharing -- the SAME filter weights apply at every position, unlike PlainFCNet's position-specific W1. But any single filter position can only ever see k=5 consecutive positions at once -- its receptive field.

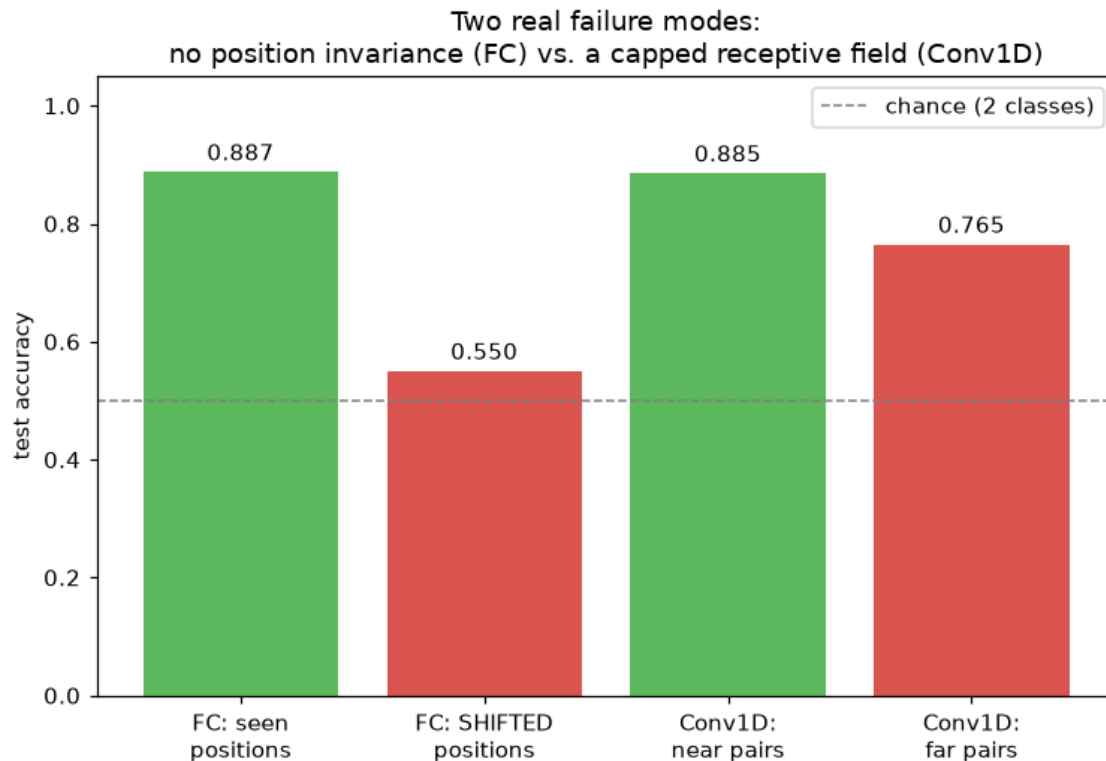
ACTUAL OUTPUT

```
overall train_acc: 0.9100 overall test_acc: 0.8033
test_acc on 'near' pairs (spike distance <= 5, n=96): 0.8854
test_acc on 'far' pairs (spike distance > 5, n=204): 0.7647
```

On 'near' pairs (both spikes within one 5-wide window at some point), Conv1DNet reaches 0.885 test accuracy -- it can detect within-window order directly. On 'far' pairs, accuracy drops to 0.765, a real and repeatable ~12-point gap. Global average pooling destroys WHICH position a channel's activation came from, blocking genuine joint order detection across a far-apart pair -- but it doesn't destroy absolute position information completely: a spike near either end of the sequence appears in fewer sliding windows than one near the middle, leaking a faint, boundary-driven positional signal the network can partially exploit. The ceiling is real, just "degraded," not "destroyed" -- softer than PlainFCNet's outright collapse.

WHY THIS CHOICE

Getting an honest reading of this result took a real correction: the first training configuration (lr=0.2, 200 epochs, C=8) left even TRAINING accuracy under 80% -- too undertrained to trust a near/far split built on it, and it produced the OPPOSITE (wrong) pattern, far pairs scoring higher than near ones. Retraining harder (lr=1.0, 800 epochs, C=16, checked directly rather than assumed) reached train_acc=0.910 and reproduced the theoretically-expected near-beats-far pattern cleanly.



4. What's Needed Instead

Both of today's networks look at the whole sequence at once (a fixed-size flatten, or a fixed-size window), then either keep position-specific weights (no invariance) or discard position via pooling (invariance, but a capped, local receptive field). What's actually needed is a network that walks through the sequence ONE STEP AT A TIME, carrying a running summary (a "hidden state") forward from every position to the next -- so information from position 2 is still available, in principle, when the network reaches position 29, and the SAME small set of weights is reused at every timestep (so sequence length is no longer fixed at construction time either). That is exactly a Recurrent Neural Network -- Day 57 builds its forward pass from scratch.

Interview Preparation -- Technical Questions

Q: Why does a dense (fully-connected) network fail to generalize a pattern across a position shift?

A: Its first-layer weight matrix has one row per input position, and each row is updated only by the gradient signal that position actually carries during training. If a position never carries useful signal in training data, its row stays near its random initialization, so the network has no learned detector there at test time -- confirmed directly in this lesson: accuracy collapsed from 0.888 (seen positions) to 0.550 (chance) on a shifted position range.

Q: Why does a 1D CNN generalize BETTER across position shifts than a dense network, but still have a real limitation?

A: A CNN's filter weights are SHARED across every position (the same k weights slide across the whole sequence), giving it translation invariance a dense network's position-specific weights lack. But any single filter application only sees k consecutive positions -- its receptive field. Two events farther

apart than k can never be jointly seen by one filter position, measured in this lesson as a real accuracy gap between 'near' (0.885) and 'far' (0.765) spike pairs.

Q: The lesson found 'far' pairs scored 0.765, well above chance (0.5), even though the theory predicted the receptive field should block order detection entirely. What explains this?

A: Global average pooling discards POSITIONAL order but not all positional information: a spike near either end of the sequence appears in fewer sliding windows than one near the middle, so its contribution to the pooled average is systematically different based on ABSOLUTE position. Since absolute position correlates with the task in a full-range dataset, the network can partially exploit this boundary effect as an imperfect substitute for true relative-order detection -- explaining why the ceiling is a real degradation, not a complete collapse to chance.

Q: Why does a dense network raise an error when given a sequence of a different length than it was built for, while a real RNN doesn't need to?

A: A dense layer's weight matrix has a fixed number of rows, one per input position -- the input size is baked into the network's parameters at construction time. An RNN instead applies the SAME small set of weights repeatedly, once per timestep, so it can process a sequence of any length by simply running for however many steps that sequence has.

Interview Preparation -- Theory Questions

Q: What does it mean for data to be "sequential," and why can that property alone break architectures that work well elsewhere?

A: Sequential data is data where ORDER carries information beyond just which values are present -- the same set of values in a different order can mean something entirely different (as today's task shows directly: identical high/low spike values, opposite labels depending on which comes first). Architectures built around treating input as a flat vector (dense) or a small local window (CNN) don't have position-independent, arbitrary-range mechanisms for tracking order across the whole input, so the failure isn't a bug -- it's a structural mismatch between the architecture and the problem.

Q: What two DISTINCT properties does an RNN need to have, based on today's two failure modes?

A: Translation invariance (the same weights apply regardless of absolute position, fixing PlainFCNet's failure) AND an unbounded effective range (information can in principle propagate from any position to any other, no matter how far apart, fixing Conv1DNet's capped receptive field) -- both achieved by processing one timestep at a time while carrying a persistent hidden state forward.

Q: Why is it important that today's lesson MEASURED both failure modes with real numbers, rather than just asserting that dense/CNN networks 'don't work' on sequences?

A: Because the two failures are different in both cause and severity -- PlainFCNet collapses to chance (0.550), Conv1DNet only degrades (0.765 vs 0.885) -- and only a direct measurement reveals that difference. Asserting a blanket 'fails on sequences' claim would hide the fact that CNNs partially work via weight sharing, information a real engineering decision (how deep/wide to make a CNN vs. reaching for an RNN) needs to know.

Key Takeaway and What's Next

Key takeaway: Sequence data breaks fixed-vector and fixed-window architectures in two distinct, measured ways: a dense network has no position invariance at all (chance-level collapse on a shifted pattern), while a 1D CNN has invariance but only within its kernel's receptive field (a real

but softer degradation on far-apart events). Both point to the same missing ingredient: a mechanism that applies identical processing at every position while letting information travel arbitrarily far across the sequence.

What's next: Day 57 builds a vanilla RNN's forward pass from scratch on a real toy sequence -- the first network in this series designed around exactly that missing ingredient.

Suggested watch order: 1) Recurrent Neural Networks, Clearly Explained!!!, 2) CNN Receptive Field.